Multiplayer Snake Game - Project Report

Natália Carvalho	Nina Cunha Pinheiro	Thiago Ramos Pinto Silva
USP: 15497232	USP: 13686500	USP: 14609446

---

1. Requirements

The project fulfills all the core requirements proposed in the assignment description, alongside custom additions tailored by our team:

- **Assignment Requirements:** LibGDX desktop project featuring a Main Menu, Instructions, and High Score screens. Core mechanics include two snakes on a grid that grow and speed up when eating food, wrap-around walls, and collision detection (self and opponent). Features 2-player controls (Arrows and WASD), real-time scoring, asynchronous sound effects, and local data persistence for the top 5 scores.
- **New Custom Requirements:** Implementation of a strict 4-color GameBoy retro palette and an arcade-style 3-letter name input for saving high-score records.

2. Project Description

The application is structured using a screen-based architecture provided by LibGDX, separating the game states (Menus, Gameplay, Game Over) into distinct classes to keep the rendering loop clean. The `GameBoard` acts as the logic hub, handling the real-time update loop (`delta` time) and managing the exact matrix positions of the snakes and the food.

The UML Class Diagram illustrates our class hierarchy, associations, and the clear separation between UI screens and backend logic. Entities like `Snake` and `Food` are pure data models using `Vector2` coordinates and `LinkedList` for body segments, allowing efficient insertions and deletions.

3. Comments About the Code

To ensure code quality and maintainability, the project emphasizes Object-Oriented Programming (OOP) best practices:

- **Inheritance & Polymorphism:** `SnakeGameMain` extends the LibGDX `Game` class, acting as a state machine. It seamlessly switches between screens because `MenuScreen`, `GameScreen`, and `GameOverScreen` all implement the polymorphic `Screen` interface.
- **Encapsulation:** Entities have their internal states strictly protected by `private` modifiers. Changes to a snake's size or direction are only possible through controlled public methods (`move()`, `grow()`), preventing external coordinate corruption.
- **Separation of Concerns:** Graphical rendering is entirely decoupled from business logic. Peripheral mechanics were isolated into managers (`ScoreManager` for file I/O Serialization, `SoundManager` for audio playback).
- **Aesthetic Implementation:** To differentiate the two players without breaking the strict 4-color palette, we applied a dithering (pixelated) effect to the second snake's rendering logic.

4. Test Plan

We adopted a hybrid testing strategy, combining automated unit testing for business logic and manual testing for visual/audio rendering.

- **Automated Unit Tests (JUnit 5):** A suite of 28 automated tests (`SnakeTest`, `FoodTest`, `ScoreManagerTest`) was created to evaluate isolated components. They verify if the snake prevents 180-degree immediate reverse turns, ensure food never spawns overlapping a snake's body, and verify if the `ScoreManager` correctly sorts and trims descending scores.
- **Manual UI/UX Tests:** Play full matches testing simultaneous keyboard inputs (P1 and P2), force head-to-head collisions to verify the "Tie" scenario, and validate asynchronous audio execution.

5. Test Results

- **Automated Tests:** `BUILD SUCCESSFUL`. All 28 JUnit 5 tests passed with a 100% success rate. We successfully utilized a headless LibGDX extension to allow mathematical testing of vectors and grid logic without opening a graphical window.
- **Manual Tests:** The game speed correctly increments up to a predefined limit. Wrap-around walls work flawlessly on all 4 borders. Audio plays without freezing the UI thread, and data serialization accurately saves 3-letter inputs. No memory leaks were detected during screen transitions.

6. Build Procedures

The project uses the Gradle wrapper, making it plug-and-play without manual IDE configuration.

Prerequisites: Java Development Kit (JDK) 17 or higher installed, and `JAVA_HOME` environment variable configured.

Step-by-Step Guide:

```
# 1. Download/Clone the repository and access the folder
$ git clone https://github.com/ninalwt/Programacao-orientada-a-objetos-2026.git
$ cd Programacao-orientada-a-objetos-2026

# 2. Run the Game (Gradle will automatically download dependencies)
$ ./gradlew run         # On Linux/Mac
$ gradlew.bat run       # On Windows

# 3. Run the Automated Tests (JUnit)
$ ./gradlew test        # On Linux/Mac
$ gradlew.bat test      # On Windows
```

7. Problems

During development, we encountered and solved the following major challenges:

- **Font Rendering:** The default `FreeTypeFontGenerator` caused `NoClassDefFoundError` exceptions. We solved this by using the Hiero tool to pre-generate bitmap fonts (`.fnt` and `.png`), eliminating external dependencies.
- **Asset Path Resolution:** When running the game via certain IDEs, the working directory couldn't find the `assets/` folder, crashing the game when loading audio. We solved this by explicitly declaring the `resources` directory and `workingDir` in the `build.gradle` configuration, ensuring cross-platform compatibility.
- **Audio Freezing the Render Thread:** We originally used `Thread.sleep()` to wait for the death sound effect to finish before changing to the Game Over screen, which caused the LibGDX render thread to freeze. We fixed it by removing the sleep method and placing the `soundManager.playDeathSound()` call inside the constructor of the `GameOverScreen`, allowing the asynchronous audio engine to play the sound perfectly while the new screen renders.

Appendix A: UML Class Diagram

The diagram below illustrates the class hierarchy, associations, and the clear separation between the UI screens and the backend logic implemented in the LibGDX framework.

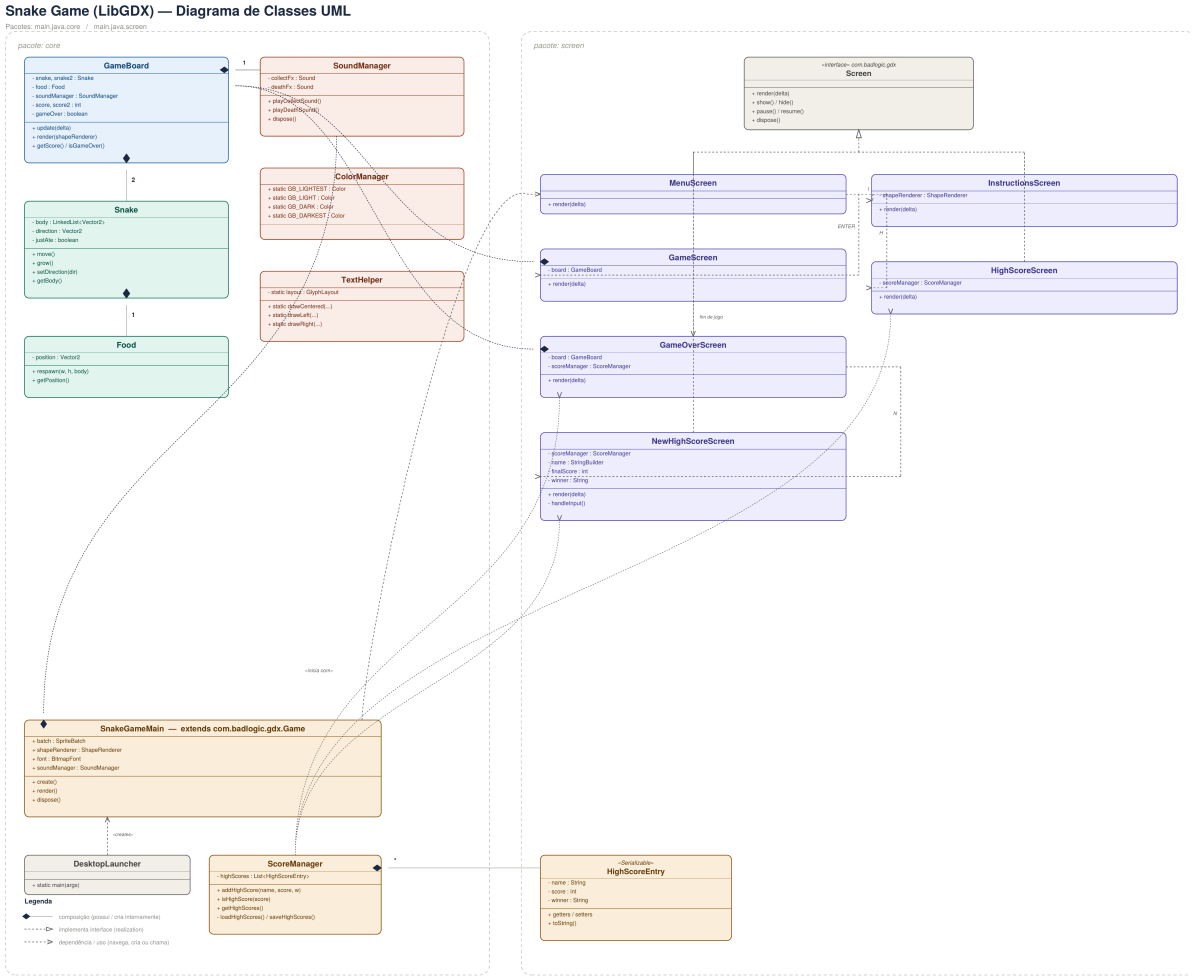

Figure 1: Multiplayer Snake Game UML Class Diagram